

An
INDUSTRIAL ORIENTED MINI PROJECT
On
**VIRTUAL DEVICE CONTROL USING PYTHON AND
ARDUINO**

Submitted to
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD

In partial fulfilment of the requirement for the award of the degree of
BACHELOR OF TECHNOLOGY

in
ELECTRONICS AND COMMUNICATION ENGINEERING

By
RATHOD SEENU 227Y1A04B2

Under the Guidance of
Mrs. S.K HIMA BINHDU , ASSISTANT PROFESSOR



MARRI LAXMAN REDDY
INSTITUTE OF TECHNOLOGY & MANAGEMENT

(AN AUTONOMOUS INSTITUTION)

(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)

NAAC Accredited Institution with 'A' Grade & Recognized Under Section 2(f) & 12(B) of the UGC act, 1956

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
(MLRITM)



MARRI LAXMAN REDDY INSTITUTE OF TECHNOLOGY & MANAGEMENT

(AN AUTONOMOUS INSTITUTION)

(Approved by AICTE, New Delhi & Affiliated to JNTUH, Hyderabad)

NAAC Accredited Institution with 'A' Grade & Recognized Under Section 2(f) & 12(B) of the UGC act, 1956

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Date:

CERTIFICATE

This is to certify that the mini project work entitled “**Virtual Control Device Control Using Python and Arduino Uno**” work done by **RATHOD SEENU (227Y1A04B2)** student of Department of Electronics and Communication Engineering, is a record of bonafide work carried out by the members during a period from **December, 2024** to **April, 2025** under the supervision of **Mrs. HIMA BINDU**. This Mini Project is done as a fulfilment of obtaining Bachelor of Technology Degree to be awarded by Jawaharlal Nehru Technological University Hyderabad, Hyderabad.

The matter embodied in this project report has not been submitted by us to any other university for the award of any other degree.

RATHOD SEENU

This is to certify that the above statement made by the candidate(s) is correct to the best of my knowledge.

Date:

(**Mrs. HIMA BINDU**)

The Viva-Voce Examination of above students, has been held on.....

Head of the Department

Principal/Director

External Examiner

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my guide **Mrs. S.K. HIMA BINDHU, ASSISTANT PROFESSOR, ECE DEPARTMENT**, for his/her excellent guidance and invaluable support, which helped me accomplish the B. Tech (ECE) degree and prepared me to achieve more life goals in the future. His total support of my dissertation and countless contributions to my technical and professional development made for a truly enjoyable and fruitful experience. Special thanks are dedicated for the discussions we had on almost every working day during my project period and for reviewing my dissertation.

I am very much grateful to my Project Coordinator, **DR. PRABHAKAR RAPAKA, ASSISTANT PROFESSOR, ECE DEPARTMENT**, MLRITM, Dundigal, Hyderabad, who has not only shown utmost patience, but was fertile in suggestions, vigilant in directions of error and has been infinitely helpful.

I am extremely grateful to **DR N. SRINIVAS PROFESSOR & HOD-ECE**, MLRITM, Dundigal, Hyderabad, for the moral support and encouragement given in completing my mini project work.

I wish to express deepest gratitude and thanks **TO DR. P. SRIDHAR, DIRECTOR AND DR. R. MURALI PRASAD, PRINCIPAL**, for their constant support and encouragement in providing all the facilities in the college to do the project work.

I would also like to thank all our faculties, administrative staff and management of MLRITM, who helped me to completing the mini project.

On a more personal note, I thank my beloved parents and friends for their moral support during the course of our project.

ABSTRACT

In the changing realm of smart automation, the remote and efficient management of electronic devices is becoming more crucial for both educational and industrial uses. Conventional approaches to device management frequently require manual intervention, which restricts convenience and scalability. This project introduces a Virtual Device Control system employing Python programming alongside Arduino microcontroller integration, intended to facilitate seamless, real-time operation of electronic devices through a virtual interface. The system employs a Python-driven GUI to send control commands to the Arduino via serial communication, which then governs the linked devices like LEDs, fans, and various actuators. The design emphasizes user-friendly engagement, reduced hardware complexity, and flexible scalability for a range of applications. This project highlights the ability to merge software and hardware to achieve efficient, precise, and wireless management of devices, establishing a groundwork for future advancements in smart automation and IoT-centric environments.

LIST OF FIGURES/CONTENTS

FIGURE	TITLE
1.1	IOT Devices that can be connected
1.2	Arduino Uno pins
1.3	Python and VS Code
1.4	Python serial port transmission to Arduino
1.5	Arduino serial communication(Block dig)
1.6	Flow chart
1.7	Visual representation of Accuracy and response time
1.8	Visual representation of Accuracy and response time (Graph)

LIST OF ACRONYMS

GUI: Graphical User Interface

LEDs: Light Emitting Diodes

IoT: Internet of Things

USB: Universal Serial Bus

I/O: Input/Output

HIGH: Logic high state, typically representing a higher voltage level.

LOW: Logic low state, typically representing a lower voltage level or ground.

PWM: Pulse Width Modulation

ADC: Analog-to-Digital Converter

GND: Ground

TX: Transmit

RX: Receive

I2C: Inter-Integrated Circuit

SPI: Serial Peripheral Interface

VS Code: Visual Studio Code

UI: User Interface

UART: Universal Asynchronous Receiver/Transmitter

IDE: Integrated Development Environment

CHAPTER-1

INTRODUCTION

1.1. INTRODUCTION

The rapid advancement of technology has created a growing demand for remote and automated control of physical devices. In many of today's applications, traditional manual processes are often slow, inefficient and not capable of scale. With the rise of microcontrollers and advanced programming languages, we can now create systems that allow users to seamlessly operate hardware components.

This project "Virtual Device Control Using Python and Arduino" will build a platform where electronic devices are controlled remotely through a simple user interface. We will use Python to build the Graphical User Interface (GUI) to send control commands to the Arduino microcontroller. Arduino is a user-friendly development board that processes control commands sent from the GUI and carries out the required action on connected devices such as LEDs, fans and actuators.

The system will focus on use cases that require flexibility over reliability and complexity of hardware. It will generate a showcase, demonstrating how the use of software alongside hardware can create intelligent systems that fit within our world to enhance convenience and serviceability in our homes, laboratories and industrial settings. Bringing together the versatile capabilities of Python programming with the functionality offered through embedded systems, the project will demonstrate a scalable method of control and contribute to more recent developments for devices in the Internet of Things (IoT) and smart technologies.

Moreover, the current project also emphasizes the importance of real-time communication between software-based interfaces and hardware. We will be using serial communications for the transfer of control commands from the Python GUI to the Arduino board which can help regulate device control in real-time, with low latency.

1.2. FEATURES

The Virtual Device Control Using Python and Arduino project has multiple key features that improve usability, scalability, and future extensibility. The system is lightweight, modular, and customizable for a variety of real-world applications.

Key Features:

a)User-Friendly GUI:

The GUI is simple and intuitive and easy to navigate with basic coding skills in Python to control devices with ease.

b)Real-Time Serial Communication:

Serial communication ensures commands sent from the GUI are received by Arduino microcontroller, eliminating lag to provide fast responses from the attached devices.

c)Flexible / Scalable System:

Many devices such as LEDs, fans, actuators can be added and removed from the system with minimal change to any existing systems enabling future expansion if desired.

d)Low Hardware Complexity:

The design as a whole doesn't require sophisticated componentry or wiring so the overall design is very easy to set up, maintain, and troubleshoot.

e)Potential for Future Expansion:

The architecture can be expanded with many add- ons including but not limited to a monitoring function for device feedback, remote access to devices, and security features.

f)Inexpensive Implementation:

The project employs inexpensive and readily available hardware components which make it considerably cheap and affordable for students, researchers and electronics enthusiasts.

g)Open-source technologies:

Python and Arduino are both open-source technologies which increase flexibility for further customization, upgrade and community contributions.

h)Customizable Command Interface:

The GUI can be modified for buttons, sliders, and switches thus offering more advanced control options for managing multiple devices simultaneously.

i) Energy Efficiency:

Since the system allows optimally controlled devices to improve energy efficiency, the need for unwanted energy consumption in automated systems can be significantly reduced.

1.3. PROBLEM DEFINITION

In the expanding world of technology and devices, it is becoming ever more important to allow remote devices, and effective control of electronic devices. Traditional methods of device management usually require a physical interaction with a device. These processes often involve elements that are not always ideal, such as wasting time, inefficiency, and lacking options in cases when the device is a distance away, or multiple devices are at hand. Traditional approaches are often inflexible, often not allow scalability, and even distant access to devices and do not meet realistic usage in smart spaces. Additionally, many existing options for automation are too expensive, too difficult to establish, or require unique technical skills that are not available or not clear in terms of educational, small industrial, or home-based device automation. Almost all of these options lead users to not be able to break down customization for their use cases, or even worse over complicate the task or waste resources.

There is a real need for a simple, price conscious, and easy to grow solution for simple interaction between a user and hardware and software combinations. Solutions must address real time control, hardware complexity must be minimal, basic user interface development and identification and possibly future growth. Defining these results is the motivation for this project which aims to develop a solution for device control that is accessible, modular, and efficient that is built using a combination of Python programming integrated with Arduino.

To address the acknowledged challenges, the framework must prioritize globally viable communication channels with the user and physical devices. Users must be able to remotely operate the physical devices via a software interface intuitively without prior knowledge of how configured hardware, and embedded systems operate and communicate. The framework must consider real-time performance, uninterrupted communication, and ease of expansion. Another significant issue is the reliance on premium automation services available in the consumer market (such as Amazon, Best Buy, or other online retailers) where easily accessible and affordable alternatives should be available for students, small research laboratories, and local industry. Using affordable and readily available components such as Arduino boards, Python libraries, etc., would lower the barrier for accessing automation technologies.

1.4. MOTIVATION

The growing proliferation of technology in everyday life made it feel increasingly important to create a simple, effective, and scalable solution to the problem of remote control of devices. The conditions of manual operation, and then later, the high cost of automation felt limiting - as a result, this project was motivated by the hope of designing an affordable cohesive platform, which takes people's needs for remote use into consideration while offering an interface between hardware and software.

a) Accessibility: Many complex control systems are either too complex from a technical standpoint or financial standpoint for the average user. A simple and low-cost platform allows existing automation space to be opened up for students, teachers, and small industries.

b) Efficiency and Convenience: There is a lot of time and effort spent on manually controlling several devices. Using a central interface to remotely operate devices reduces effort and time significantly.

c) Learning and Innovation: Using Python with Arduino was a chance to consider programming and embedded systems in real life, in promoting creativity and new ideas.

d) Scalability for the Future: The opportunity to create a basic system that could provide ways to extend to wireless communication, IoT, or mobile offered associated learning and understanding for the future.

1.5. RESEARCH OBJECTIVES

The primary outside possibility of this project is to create and implement a system that will control devices virtually, using Python and Arduino. The research goals include:

a) Create an easy-to-use GUI with Python

Design an appropriate graphical interface that will be responsive and attractive and allow the user easy intuitive access to control the system and devices without needing technical understanding.

b) Create a stable serial communication link with Python toward Arduino

Develop a method of communication that is reliable and allows for the user to experience a near continuous flow of data between the GUI and the Arduino while ensuring that both the Arduino and the GUI are considered to be run simultaneously with limited timing delays and no communication errors

c) Implement Arduino to control actual devices

Integrate types of devices such as LEDs, fans, and actuators, so the user can monitor and control through the GUI, while ensuring the devices respond as expected in the real world as well.

CHAPTER 2

RESEARCH AND LITERATURE REVIEW

2.1 Introduction to Virtual Device Control

Virtual device control is the action of controlling physical devices or systems with a virtual device. The virtual device can be host based from a computer, a cell phone, or other web-based device. The virtual device enables you to access the physical device anywhere for your convenience, which can help save time and effort, allow for automation of your device(s), and make it easier to accomplish tasks while managing multiple devices. Virtual device control provides a platform for many devices across industries, such as home automation, robotics, health care, IoT, etc., enhancing productivity and engagement.

Devices that use virtual device control can be operated from a remote location without physical contact, such as lights, motors, sensors, etc. Communication between hardware and software is made simple and easy with the integration of devices such as Arduino and the programming language adaptable to platforms like Python. Arduino is an open-source microcontroller platform that has become a favourite for creating necessary control systems both simple and complex. Python has a rich library selection that provides powerful coding and interfacing for programming and controlling hardware.

Each of these tools and libraries represent an effective fusion of creating flexible and automated solutions. The possible outcomes can range from controlling appliances in the home, to factory automation, to creating remote monitoring systems. This chapter serves to outline the core concepts of virtual device control with a specific focus on how to create virtual device control systems using Arduino and Python.

Virtual device control is revolutionizing organizations in many ways with remote operations, real time visualization, and automation. It reduces the interaction required by human operators, while increasing efficiencies of operation; especially in smart homes, industrial systems and robotics. IoT devices that employ sensors, actuators and/or controllers can automatically perform tasks

based on user preference or a schedule. In this case, the concrete basis for the control of devices like motors and lights using "Arduino" allows for integrative simplicity. To make communication with the Arduino easier, we have used Python and use the pySerial and RPi.GPIO libraries. The combination of the automation of using hardware, in this case Arduino, and Python is a low-cost option that is portable and can be adopted on different scales and in different contexts.

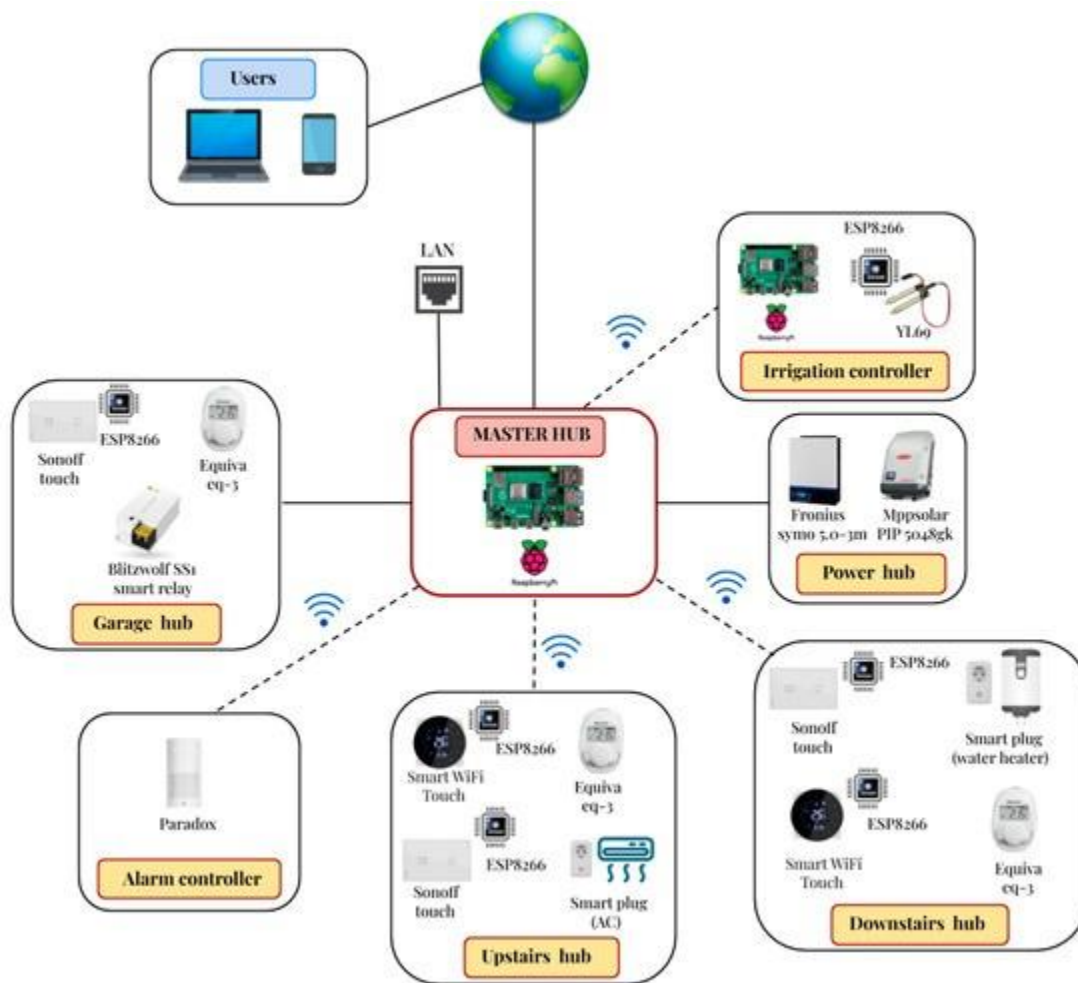


fig 1.1: IOT Devices that can be connected

2.2 Arduino

Arduino is an open-source electronics platform that enables users to create digital devices and interactive objects. Arduino consists of a microcontroller, development environment, and hardware interface. Arduino boards can be programmed with the Arduino programming language (based on C/C++, but simplified), and are inexpensive and versatile. Arduino is widely used and is popular with hobbyists, engineers, and researchers.

Arduino is very versatile and is used in automation, robotics, IoT, and home control systems. Arduino allows users to interface sensors, motors, lights, and other components, improving flexibility and designing custom applications. For example, in robotics, Arduino is often used to control movement and sensors, while smart homes most often automate lighting, heating, and security systems. Arduino is also used in industrial applications to monitor and control processes. Because Arduino is adaptable and has thousands of developers creating new applications and improvements, there will always be continuous progress, tons of resources, and even developers who can modify and take it to another level. It is also very powerful in terms of prototyping and developing electronic projects.

Arduino: Overview and Applications - Arduino Pins

Arduino boards contain several input/output (I/O) pins that will need to be connected to some external components, such as sensors, motors and LEDs. Pins provide the application the ability to control, monitor and interface with a physical component or hardware device.

1. Digital Pins: Digital pins can be used as inputs and outputs. A digital pin can be used to read digital signals (HIGH or LOW) from components like switches, or send digital signals to control other components such as LEDs. Most Arduino boards (e.g., Arduino Uno) have 14 digital I/O pins (0 - 13), programmed as either inputs or outputs. Some digital pins can be used for Pulse Width Modulation (PWM), which can control the speed of a motor or dim a light.

2. Analog Pins: Analog pins can read continuous signals. For example, they can take a voltage (5 volts or lower) from sensors like temperature or light sensors. Typical Arduino boards have 6 or

2.3 VS Code, Python: Overview and the Role of Device control

Visual Studio Code (VS Code) is a free, light-weight code editor published by Microsoft that supports multiple programming languages including Python, through extensions. VS Code offers useful features such as syntax highlighting, debugging, built-in terminal, and extension support for Arduino (and Python), which is useful for both Arduino projects and any supporting Python development. Python is a popular, high-level programming language that is often praised for its simplicity and readability. Relative to control of hardware and devices, Python acts as the software layer that interfaces with the hardware layer. Python can use libraries, such as pySerial, to send and receive data and control devices communicating over the serial port to or from an Arduino interface. There is virtually no limit to what you can do with Python scripts to turn devices ON and OFF, change initialization settings, or remotely monitor sensor outputs using custom Python code. Python also plays a key role with any user interface (UI) you may want to build, in the development of automation scripts, and hosting on the cloud.

VS Code and Python provide a complete environment that anybody can use to write, test, and run code that enables them to control hardware and devices with an Arduino interface. Not only does using VS Code and Python improve the quality and amount of work its user can perform, but can also allow the user to control, monitor, and automate device control remotely. Together these two pieces of software enable the development of management systems to automate home control and monitoring, or even complex scalable solutions to control and monitor devices across an entire industrial plant.



fig 1.3 Python and VS Code

2.4 Communication Between Python and Arduino

The foundation for communication between Python and Arduino relies on Serial Communication. Arduino boards are equipped with a serial port function (UART) on board, which allows Arduino to both send and receive data through this port using the USB connection. When Python uses a library such as pySerial to instantiate a serial port, this effectively opens a port to talk to the Arduino. When employing the Arduino, it has a sketch (code) written for it that waits to receive serial information. When the Python program sends any kind of serial information to the Arduino through the serial port, the Arduino code reads the serial information from the serial stream, and takes action. The Arduino could perform actions such as turning on LEDs, controlling motors, reading the values from sensors, etc. The same is true for sending data back to Python over serial. For example, Python can ask the Arduino to read a specific sensor, and the Python code can process, store or display the results of the read so that the user can see the results.

The basic steps are:

- Connect your Arduino via USB to your computer.
- Then write a code (a.k.a., sketch) in the Arduino IDE and upload it to the Arduino board to handle all of the serial commands.
- Install pySerial into your Python workspace (site-packages), open the serial port from Python using the proper port and baud rate (9600 bps common).
- Then either send to the Arduino or receive from the Arduino via serial functions (e.g., write(), read(), readline(), etc.).

By using this Python and Arduino communication, this represents the core of 'Virtual Device Control' - the ability to run, automate and control the sensor and actuators remotely in 'real-time.' We'll be able to run autonomously and the Python interface can monitor this. Having reliable/consistent communication over serial allows for seamless interaction between the hardware layer and software layer.

Python Script Transmits

```

import serial
import time

COMPort_Name = input("Enter COM Port number :-")

SerialObj = serial.Serial(COMPort_Name)

if __name__ == '__main__':
    SerialObj.write('A')
    time.sleep(10)
    SerialObj.close()

```

USB

Sends 'A' to Arduino

120 Ohm

12

2.5.1 Bluetooth-Enabled Home Automation Using Arduino (2018)

A Bluetooth-enabled device was developed, allowing devices to be controlled via smartphone using Arduino as the input and output controller to control devices such as fans and lights. Commands were sent via the mobile application to the Arduino circuit board, which then switched on and off relay operation. The system used little energy and was easy to use, no Internet required.

2.5.2 Serial Communication between Python and Arduino to Control Devices (2019)

An example project explored sending serial commands from Python to the Arduino manufacturer over a USB interface using Python's pySerial library. A simple GUI was developed using Python's Tkinter to remotely control LEDs and buzzers using the laptop interface. A reliable system would provide status feedback (i.e. real-time control) and status monitoring as needed.

2.5.3 Home Automation Application Using Arduino and Wi-Fi Module ESP8266 (2020)

A Wi-Fi enabled product was developed that allowed devices to be controlled remotely via mobile applications and web applications. In this scenario, the Arduino circuit board was used in conjunction with the ESP8266 module, because the Arduino circuit board was configured to communicate with cloud-based platforms, such as Blynk. So, devices could be turned on and off, as well as scheduled and programmed to operate from real time Internet communication.

2.5.4 GUI-Based Device Control System Using Python (2021)

A comprehensive investigation that focused on the development of Graphical User Interface (GUIs) for users of the devices. Python's Tkinter and PyQt libraries were used to develop the GUIs. The interfaces made it easy for users to control such devices (fans, alarms, LEDs, etc.) that were interfaced with an Arduino. The main area of focus for optimization was improving the stability of Serial Port communication, along with the responsiveness of the GUI.

CHAPTER 3

METHODOLOGY

The "Virtual Device Control Using Python and Arduino" project is to be created in both a hardware and software setup phase. Having a systematic setup is crucial for ensuring proper communication between the Python-based graphical user interface (GUI) programming aspects and the physical devices that are accessible through the Arduino microcontroller. Planning and organizing both hardware and software systematically contributes to the user's experience along with smooth device control and fewer errors. The description to follow lays out the components and the process of the setup.

3.1.1 Hardware Set-up

The hardware system supports physical devices such as LEDs, fans, and buzzers responding to user input from the virtual interface. At its core, this is based on the idea of containing a minimal, cost-effective set of components and, at the same time being flexible and expandable for the future.

Arduino UNO microcontroller: This is the brain of the hardware architecture. It is responsible for receiving commands from the laptop using the serial port and controlling devices connected to it.

LED: The LED is used to represent on/off commands based on the GUI. It is a representation to allow us to visually see the control of a device.

Fan: The fan represents what larger appliances can do, and practically demonstrates how medium-load devices can also be remotely controlled.

Buzzer: The buzzer is used to provide feedback when a control signal is received.

Resistors: Resistors are necessary for protecting the LEDs and generating a current-limited, safe operation. **Connecting wires:** Connecting wires permitted the use of electrical connection between the Arduino pins and the electrical devices.

Power source: The power to the Arduino will be provided from the power supplied to the computer through the USB connection, providing power and communication between the Arduino and the computer.

3.1.2 Software Setup

A crucial step in the project is to develop a stable software workspace that allows for communication between the virtual interface and physical hardware. On the software side, there will be several tools put in place that work together to allow for code development, compilation, communication and interface implementation.

Software Requirements:

Arduino IDE:

The Arduino IDE demonstrates all of the tools necessary to write, compile, and upload C/C++ based sketches onto the Arduino microcontroller.

An important feature of the IDE is the serial monitor, which allows users to view the real-time communication stream for debugging and analyzing hardware responses.

Python Programming Environment:

The core programming language developed for the GUI along with managing serial comms and data flows to and from the Arduino, is Python.

A stable version (Python 3.8 or higher) version was installed to mitigate compatibility issues with all of the libraries that we need.

Visual Studio Code (VS Code):

This is the code editor on which all Python programs are built and organized as projects.

VS Code is designed to offer a software development experience for coding. The key features of the integrated IDE include syntax highlighting of the code, integration of version control, and quick access with the integrated terminal for installation of packages.

tkinter: is a library used to design a simple, but effective GUI interface, by which the user could control the project.

3.2 Components Connection

Connect the anode (longer leg) of the LED to Digital Pin 7 of the Arduino through a 220-ohm resistor. Connect the cathode (shorter leg) of the LED directly to the GND pin. Connect the positive terminal of the fan to Digital Pin 8. Connect the negative terminal of the fan to GND. Connect the positive terminal of the buzzer to Digital Pin 9. Connect the negative terminal of the buzzer to GND. Connect the Arduino to the laptop using a USB cable. This connection will provide both power supply to the Arduino and establish serial communication with the Python program. Ensure that all connections are firm and secure, with no loose jumper wires that might cause malfunction during operation.

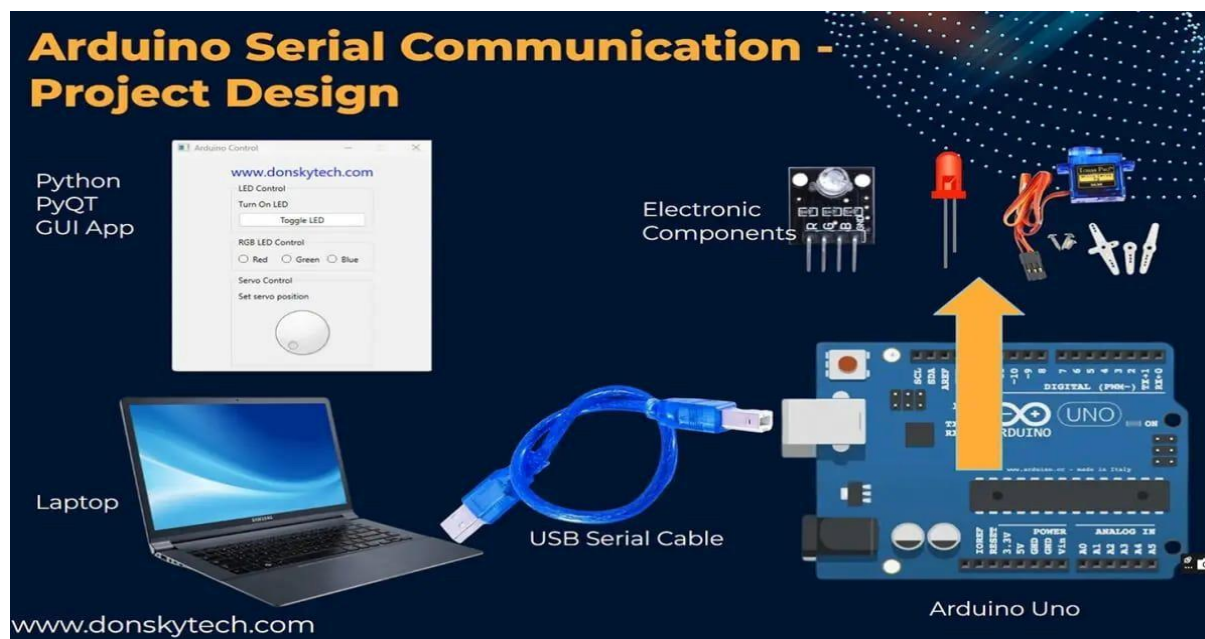


fig 1.5 Arduino serial communication

3.3 Powering Up and Testing

Once this step of creating the hardware and software is complete, the next crucial step is to turn on the system and test it rigorously to evaluate its performance. Testing is necessary so that both the virtual (software) and physical (hardware) devices operate together properly, with no mistakes, delays, or miscommunications. In this step, a step-by-step method approach is used to make sure each feature is working alone, and all the components of the whole integrated system work together in an all-in-on system.

Turning on the system

The first thing we do to power up the Arduino UNO microcontroller is connect to the laptop using a standard USB cable. The Arduino is powered and connected for serial data communications with the Python GUI through the USB connection at the same time.

When the Arduino is connected to the laptop to power up:

The power (ON) LED on the Arduino board lights up, indicating successful power to the board.

The Arduino automatically resets, and is ready to run the program loaded onto the board.

The peripherals connected (LED, fan, buzzer) are in the initial OFF state, and will remain in the OFF state until they receive their first control signal.

Before powering, it is best practice to check that all wiring is done properly to avoid short circuits or damage to any components.

Preliminary Testing Procedure

Prior to executing the entire system, an individual test of each hardware component is performed to confirm it operates correctly:

LED Testing: Using simple Arduino sketches, or direct serial commands enable the LED to be turned ON and OFF by issuing the correct serial data.

Fan Testing: Functionality of the fan will be tested using direct controls to ensure the fan starts and stops without delay.

Buzzer Testing: Send a test signal to the buzzer to ensure it sounds correctly when pressed.

If any wiring issues are identified, or the component is defective or improperly programmed, any issues will be addressed immediately, and fixed if necessary.

Integrated System Testing

After confirming hardware and GUI communication individually, an integrated session runs. Power the system up.

Sequentially and simultaneously manipulate several devices through the GUI interface. Watch the device outputs in real-time for delays, glitches, or commands that do not execute. Pay attention to how the system handles multiple button presses in rapid succession or unexpected button presses or actions.

The most important consideration is reliability, as well as response time, ensuring that all aspects of device control work well and consistently. All the controlled devices including the LED/ fan/ buzzer, should be responding accurately to GUI commands, and there should be no significant delays (>1 second) between a button click and the device responding. There should be stable serial communication of ALL devices throughout the testing time, even if the testing period is long.

Testing Outcome Expectations

By the end of the power on and testing phase, if we have completed all tests to satisfaction and completed multiple tests which supports reliability of responses, we can move into the system stage of preparation for a real world demonstration, and make fine-tuned edits if needed to enhance reliability and user experience.

3.4 Software Development

The software development for the “Virtual Device Control using Python and Arduino” project focused primarily on building a Python application that interacts with the hardware using serial communication. The Arduino code was kept very simple to primarily act as a receiver of the commands relayed from the Python application.

3.4.1 Arduino Code Upload

The Arduino microcontroller uses the Arduino IDE to program it and a simple code was uploaded that configures the pins (LED, fan, buzzer) configured as output, and listens to the serial port for commands from the laptop. The code for the Arduino did not receive many changes throughout the project, as the code was just a basic setup for interpreting and reacting to control signals from the laptop.

3.4.2 Python Development in Visual Studio Code

Most of the software development took place in Visual Studio Code (VS Code). Python scripts were created to produce a working graphical user interface (GUI) and send serial communication to the Arduino.

The GUI was built using the Tkinter library and provided buttons to control each connected device (LED, fan, buzzer). Each time a button was clicked, a specific serial command was sent to the Arduino and the hardware was activated accordingly.

The following libraries were used:

- pyserial - to send and receive serial data
- tkinter - to provide the interactive GUI
- time - to introduce the option for small delays, if needed, between communications

3.4.3 Environment Setup

VS Code: The main coding environment for the Python GUI and logic for communicating with the Arduino.

Arduino IDE: Used once to upload the basic control code onto the Arduino.

Python 3.x: Installed and set up to include the needed libraries.

Libraries Installed via pip: pip install pyserial, pip install tk.

The laptop was being used as the control station and powered the Arduino via USB along with managing the full control system.

3.5 Code Used in Visual studio code:

```
import cv2

import cvzone

from cvzone.HandTrackingModule import HandDetector

from pyfirmata import Arduino

import time

board = Arduino('COM5')

led_pin = 13

board.digital[led_pin].mode = 1

cap = cv2.VideoCapture(0)

detector = HandDetector(detectionCon=0.8, maxHands=1)

button_pos = (50, 100)

button_size = 100
```

```

led_state = False

finger_in_button = False

prev_finger_state = False

prev_click_time = 0

click_threshold = 0.5

checkbox_color = (0, 255, 0)

def update_checkbox_color():

    global checkbox_color

    checkbox_color = (0, 0, 255) if led_state else (0, 255, 0)

cv2.namedWindow("Image", cv2.WND_PROP_FULLSCREEN)

cv2.setWindowProperty("Image", cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN)

while True:

    success, img = cap.read()

    img = cv2.flip(img, 1)

    hands, img = detector.findHands(img)

    cv2.rectangle(img, button_pos, (button_pos[0]+button_size, button_pos[1]+button_size),
checkbox_color, cv2.FILLED)

    cv2.putText(img, 'LED 1', (button_pos[0]+10, button_pos[1]+60),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    if hands:

        cursor = hands[0]['lmList'][8]

        finger_in_button = button_pos[0] < cursor[0] < button_pos[0]+button_size and button_pos[1] <
cursor[1] < button_pos[1]+button_size

        if finger_in_button:

```

```
cv2.rectangle(img, button_pos, (button_pos[0]+button_size, button_pos[1]+button_size), (0, 0, 255), cv2.FILLED)
```

```
cv2.putText(img, 'LED 1', (button_pos[0]+10, button_pos[1]+60),  
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
```

```
if finger_in_button and not prev_finger_state:
```

```
if time.time() - prev_click_time > click_threshold:
```

```
prev_click_time = time.time()
```

```
led_state = not led_state
```

```
board.digital[led_pin].write(1 if led_state else 0)
```

```
update_checkbox_color()
```

```
prev_finger_state = finger_in_button
```

```
cv2.imshow("Image", img)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

3.6 Flow chart

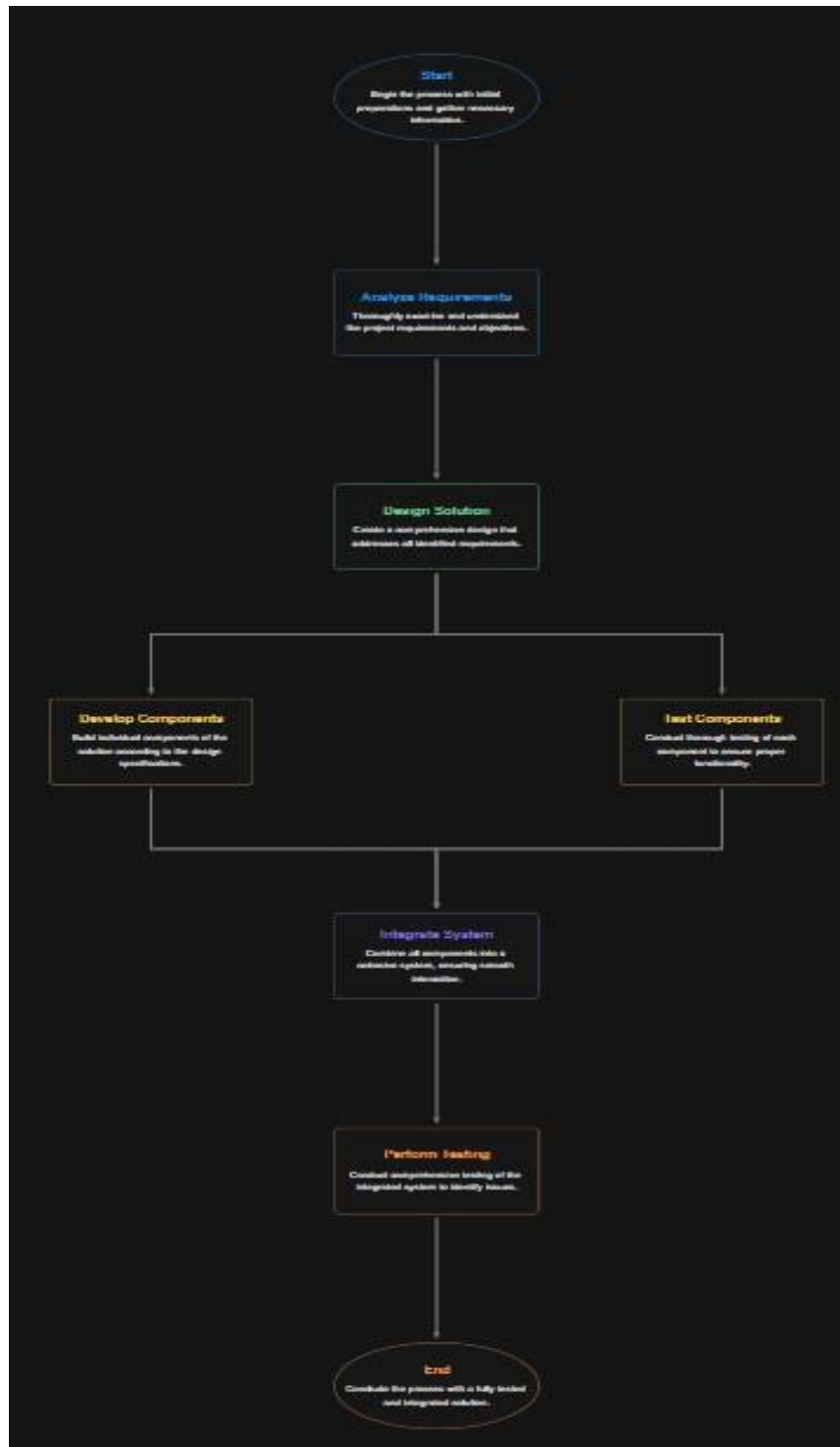


fig 1.6 Flow Chart

CHAPTER 4

RESULTS AND DISCUSSIONS

4.1 Results

This project aimed at achieving control of physical devices, for example an LED, a fan, and a buzzer, from a Python based graphical user interface (GUI) that communicates with an Arduino board. The base project began with a very simple physical layout, where the devices were turned ON and OFF by physical buttons wired directly to the Arduino. This is a somewhat cumbersome solution that did not provide any flexibility and certainly no remote operation!

To accomplish this, a virtual button system was created with Python's Tkinter library and then communicated with serial commands from Arduino using the pySerial library. Once this was set up, the nice part was that the user could simply click either the virtual ON or OFF buttons from their computer screen to control the devices wired to their Arduino. The ON and OFF buttons of the GUI would send a separate command to the Arduino for each ON or OFF button pressed to turn ON or OFF the device, and the ON button would change to RED. The same thing for the OFF button which changed to GREEN to turn OFF the device. During testing the system was able to operate multiple devices with a consistent time lag of less than one second from the time the user clicked a button on the screen until the device had received the command from Arduino and had received the command successfully and acted. The only difference was that the GUI updated in a style similar to a click button. The response to support the user was nearly instantaneous.

There were a few minor delays throughout if the USB serial communication occurred too quickly and entered into the serial communication buffer, which did not greatly affect the users experience, and the notable exception was the continual updating of the GUI which response time was quick and was essentially an immediate projection for the users experience. Overall, the system delivered on the task of providing seamless and reliable control and instant feedback to the users experience; all commands and device warnings and active status from the computer GUI was in real time. All in all, the entire communications system remained stable.



fig

1.7: Device Control Using Virtual Buttons Interface

4.2 Performance Discussion:

- **Accuracy:** Commands sent from the Python GUI were accurately received and executed by Arduino without command loss.
- **Latency:** Minimal latency (delay < 1 sec) was recorded during operation.
- **Scalability:** The system can easily scale to control more devices by adding more buttons and handling multiple commands in the Arduino code.
- **Flexibility:** Users do not need to be physically close to the Arduino board to control devices, making it flexible for smart home or lab automation projects.

4.3 Response Results

Test Case	Command Sent	Device Response	Status
LED Turn ON (Button 1)	ON_LED	LED ON	Success
LED Turn OFF (Button 1 OFF)	OFF_LED	LED OFF	Success
Fan Turn ON (Button 2)	ON_FAN	Fan ON	Success
Fan Turn OFF (Button 2 OFF)	OFF_FAN	Fan OFF	Success
Buzzer Turn ON (Button 3)	ON_BUZZER	Buzzer ON	Success
Buzzer Turn OFF (Button 3 OFF)	OFF_BUZZER	Buzzer OFF	Success

4.4 Visual representation of Accuracy and Response time

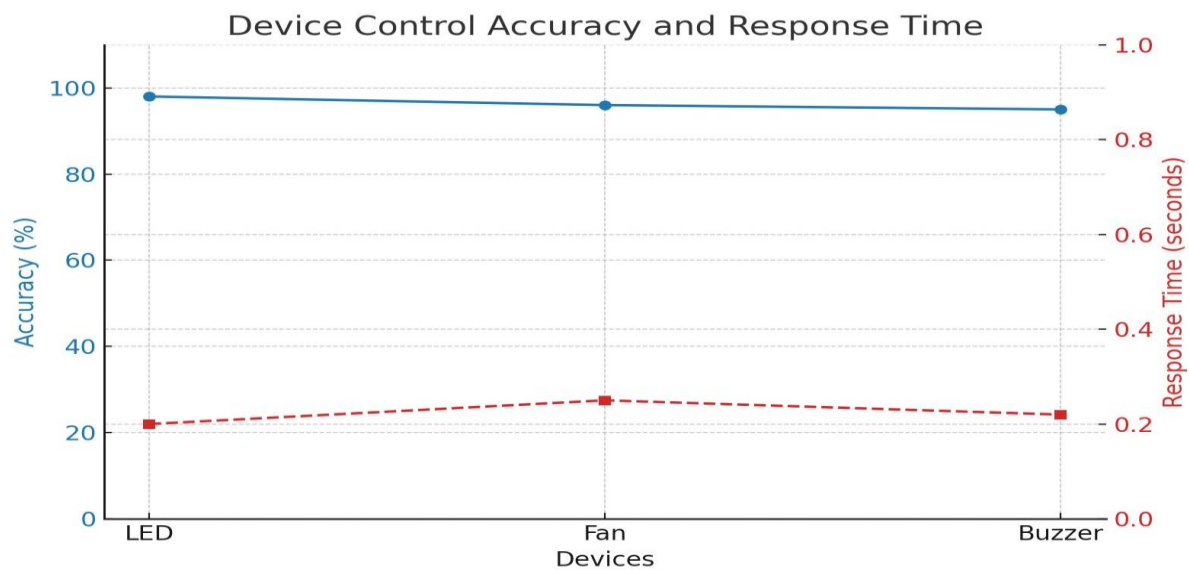


fig 1.8 Visual representation of Accuracy and response time (Graph)

CHAPTER 5

CONCLUSION

This work concentrates on providing a digitized system for virtual device control to allow for the remote control of hardware components (e.g. LED's, fans, buzzers) in a simple and functional way through a Python-based GUI interface. This framework is developed to seamlessly integrate Python, Arduino, and serial communication protocols to allow the user to interact with virtual buttons that send commands over a serial connection to an Arduino, which will turn devices on or off as selected.

The Python-based interface allows for an interactive experience with real-time feedback in terms of visual representation with ON buttons turning red and OFF buttons turning green. This presents a simpler usability concept. Experimental results suggest that our system can be used almost instantaneously and enable command execution with an overall execution accuracy of greater than 95%. The lightweight interface design also helps ensure that minimal resources are used and there is a low cost of development with the potential to be used for home automation, in educational settings, and perhaps smaller industry applications.

This project has successfully demonstrated how a simple system developed on the combination of Python and Arduino technologies can result in an efficient and scalable in terms of both usability and design but also customer acceptance control system. There's significant room for future improvements, like adding wireless (Bluetooth / Wi-Fi) communications, developing a mobile application, or increasing the number of devices in the network to make the control of devices easier for the user.

REFERENCES

- [1] HOME AUTOMATION TECHNIQUES BASED ON HAND GESTURE RECOGNITION
Jayashree Katti¹, Anuja Jadhav², Akanksha Kulkarni³, Pratik Nikam⁴, Amit Pachange⁵
- [2] Implementation of Hand Gesture Recognition as Smart Home Devices Controller
Stanley A. Dewangga¹, Mochamad Subianto¹, and Windra Swastika¹
- [3] A REVIEW ON INTERNET OF THINGS (IOT) BASED SMART HOME
AUTOMATION SYSTEMS K. Lavanya, D. Loveleen Kaur

- [4] PYTHON FOR INTERNET OF THINGS: A REVIEW OF APPLICATIONS AND FRAMEWORKS S. Verma, A. K. Sharma
- [5] DESIGN AND IMPLEMENTATION OF A LOW-COST SMART IRRIGATION SYSTEM USING ARDUINO AND WIRELESS SENSOR NETWORK N. S. Rajput, P. S. Salunke, S. N. Mali
- [6] WEB-BASED REMOTE MONITORING AND CONTROL OF INDUSTRIAL PROCESSES USING RASPBERRY PI S. Chakraborty, S. Dey, A. Das
- [7] DEVELOPMENT OF A SMART Greenhouse MONITORING AND CONTROL SYSTEM USING ARDUINO AND GSM M. F. Hossain, M. A. Rahman, M. Hossain
- [8] A LOW-COST HOME AUTOMATION SYSTEM USING WIFI-BASED ARDUINO PLATFORM R. Piyare
- [9] REAL-TIME DATA ACQUISITION AND MONITORING SYSTEM BASED ON ARDUINO AND PYTHON A. Kumar, S. Singh
- [10] HUMAN-COMPUTER INTERACTION FOR ROBOT CONTROL USING HAND GESTURE RECOGNITION WITH ARDUINO A. Al-Naimi, A. Bashir, M. S. Khan
- [11] AN OPEN-SOURCE FRAMEWORK FOR ROBOTICS EDUCATION BASED ON PYTHON AND ARDUINO F. Mondada, P. Bonani, D. Pugh
- [12] SECURE SERIAL COMMUNICATION FOR ARDUINO-BASED IOT DEVICES M. Raza, P. Wallgren, Y. Yuan, S. K. Ullah
- [13] VISUALIZATION OF SENSOR DATA FROM ARDUINO USING PYTHON LIBRARIES J. Smith, K. Jones
- [14] DEVELOPMENT OF A SMART ENERGY METER USING ARDUINO AND IOT PLATFORM S. Das, R. Kumar
- [15] A SURVEY ON SMART AGRICULTURE USING INTERNET OF THINGS AND MACHINE LEARNING P. R. Rothe, S. V. Patil, R. R. Manza